

```
$ lab --phase 0 --num 1
```

Lab 1

Linux Hardened Server

Mise en place complète d'un serveur Ubuntu sécurisé

01 Réseau	02 Users	03 SSH	04 UFW	05 Fail2Ban	06 Systemd	07 Logrotate
--------------	-------------	-----------	-----------	----------------	---------------	-----------------

Système	Auteur	Date	Statut
Ubuntu Server 22.04 LTS	Nathan	Juin 2026	Validé ✓

Procédure complète de mise en place d'un serveur Ubuntu durci pour la production. Configuration réseau, durcissement SSH, firewall UFW, protection brute-force Fail2Ban, service d'audit systemd et rotation des logs.

Table des matières

01 Configuration réseau (Netplan)

- › IP statique
- › Résolution DNS

02 Utilisateurs et droits

- › Utilisateur sudo
- › Utilisateur deployer

03 Sécurisation SSH

- › Changement de port
- › Authentification par clé
- › MaxAuthTries

04 Firewall UFW

- › Règles par défaut
- › Ports ouverts
- › Activation

05 Protection brute-force (Fail2Ban)

- › Installation
- › jail.local
- › Intégration UFW

06 Service systemd d'audit

- › Script Python
- › Fichier .service
- › Activation

07 Rotation des logs (Logrotate)

- › Configuration
- › Validation

08 Validation finale

- › Checklist complète

CHAPITRE 01

Configuration réseau (Netplan)

Sur Ubuntu Server 22.04, la configuration réseau se fait via Netplan, qui génère ensuite la configuration pour systemd-networkd. Objectif : configurer une IP statique pour que le serveur soit toujours joignable à la même adresse sur le réseau local.

1.1 Éditer la configuration Netplan

```
sudo nano /etc/netplan/00-installer-config.yaml
```

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 192.168.1.3/24
      routes:
        - to: default
          via: 192.168.1.254
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

■ Erreur fréquente

Sur Ubuntu Server, utiliser `renderer: networkd` (et non `NetworkManager`). `NetworkManager` est pour les machines desktop. Avec `NetworkManager` configuré, `netplan apply` échoue avec `'NetworkManager.service not found'`.

1.2 Appliquer la configuration

```
sudo netplan generate
sudo netplan apply

# Vérifier l'IP attribuée
ip a

# Tester la connectivité et la résolution DNS
ping -c 3 8.8.8.8
ping -c 3 archive.ubuntu.com
```

■ À savoir

Un warning 'Cannot call Open vSwitch: ovsdb-server.service is not running' peut apparaître. C'est normal et sans conséquence si Open vSwitch n'est pas utilisé.

CHAPITRE 02 Utilisateurs et droits

Bonne pratique de sécurité : ne jamais se connecter directement en root. On crée un utilisateur sudo pour l'administration et un utilisateur dédié sans shell pour les déploiements futurs.

2.1 Utilisateur administrateur (user)

Créé pendant l'installation d'Ubuntu Server avec droits sudo.

```
bash
# Vérifier que user est bien dans le groupe sudo
groups user

# Doit afficher : user sudo
```

2.2 Utilisateur deployer (sans shell)

Utilisateur dédié pour les futurs déploiements automatisés. Pas de shell signifie qu'il ne peut pas se connecter interactivement — c'est de la défense en profondeur.

```
bash
# Créer l'utilisateur sans home et sans shell
sudo useradd --no-create-home --shell /bin/false deployer

# Vérifier que le shell est bien désactivé
grep deployer /etc/passwd

# Affichage attendu : deployer:x:1001:1001::/home/deployer:/bin/false
```

■ Pourquoi sans shell ?

Un utilisateur avec `/bin/false` comme shell ne peut pas ouvrir de session SSH ni de terminal. Il peut uniquement être 'utilisé' par des services systemd. Si compromis, l'attaquant n'a aucun accès interactif.

CHAPITRE 03

Sécurisation SSH

SSH est la porte d'entrée principale du serveur — c'est aussi la cible principale des attaques. Trois durcissements essentiels : changer le port par défaut, désactiver l'authentification par mot de passe et limiter les tentatives par connexion.

3.1 Modifier la configuration SSH

```
bash
sudo nano /etc/ssh/sshd_config
```

Paramètres à modifier :

```
ini
# Changer le port par défaut (réduit drastiquement les scans automatisés)
Port 222

# Désactiver la connexion root directe
PermitRootLogin no

# Désactiver l'authentification par mot de passe
PasswordAuthentication no

# Limiter les tentatives d'auth par connexion
MaxAuthTries 2
```

3.2 Ajouter sa clé SSH

```
bash
# Depuis votre PC (Windows/Linux/Mac) :
ssh-keygen -t ed25519 -C "nathan@laptop"
ssh-copy-id -p 22 user@192.168.1.3

# Sur le serveur : vérifier que la clé est bien ajoutée
cat ~/.ssh/authorized_keys
```

3.3 Redémarrer SSH

```
bash
sudo systemctl restart ssh
sudo systemctl status ssh
```

■ Règle absolue

Ne JAMAIS fermer la session SSH active avant d'avoir testé la nouvelle config depuis une autre session. Si une erreur empêche la connexion, on garde un accès pour corriger le problème. Tester avec :

```
ssh -p 222 user@192.168.1.3
```

CHAPITRE 04 Firewall UFW

UFW (Uncomplicated Firewall) est le gestionnaire de firewall par défaut d'Ubuntu. Il simplifie la gestion d'iptables avec des commandes lisibles.

4.1 Installation et règles par défaut

```
• • • bash
sudo apt install ufw -y

# Tout bloquer en entrée, tout autoriser en sortie
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

4.2 Ouvrir les ports nécessaires

```
• • • bash
# Nouveau port SSH (AVANT d'activer UFW)
sudo ufw allow 222/tcp

# Bloquer l'ancien port SSH
sudo ufw deny 22/tcp

# HTTP et HTTPS pour les futurs services web
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

■ ■ Ordre critique

TOUJOURS autoriser le nouveau port SSH AVANT d'activer UFW. Sinon, ufw enable coupera votre session SSH actuelle (port 22) et le nouveau port (222) ne sera pas encore ouvert.

4.3 Activer et vérifier

```
• • • bash
sudo ufw enable
sudo ufw status numbered
```

Résultat attendu :

```
bash
Status: active

      To          Action      From
      --          -
[ 1] 22/tcp       DENY IN    Anywhere
[ 2] 80/tcp       ALLOW IN   Anywhere
[ 3] 443/tcp      ALLOW IN   Anywhere
[ 4] 222/tcp      ALLOW IN   Anywhere
```


CHAPITRE 05

Protection brute-force (Fail2Ban)

Fail2Ban surveille les logs SSH et bannit automatiquement les IPs qui effectuent trop de tentatives de connexion échouées. Avec UFW actif, Fail2Ban doit utiliser UFW comme action de ban pour que les règles soient correctement appliquées.

5.1 Installation

```
sudo apt install fail2ban -y
sudo systemctl enable fail2ban
```

bash

5.2 Configuration via jail.local

Ne jamais modifier jail.conf — créer jail.local qui surcharge :

```
sudo nano /etc/fail2ban/jail.local
```

bash

```
[DEFAULT]
ignoreip = 127.0.0.1/8 192.168.1.0/24
bantime  = 1h
findtime = 10m
maxretry = 3
backend  = auto

[sshd]
enabled  = true
port     = 222
logpath  = %(sshd_log)s
backend  = %(sshd_backend)s
maxretry = 3
bantime  = 2h
banaction = ufw
```

ini

■ ■ Point critique

banaction = ufw est CRITIQUE quand UFW est actif. Sans ça, Fail2Ban tente d'écrire des règles iptables qui sont ignorées par UFW, et les bans ne bloquent rien.

5.3 Vérifier /etc/fail2ban/jail.d/defaults-debian.conf

Le fichier doit contenir uniquement :

```
[sshd]
enabled = true
```

ini

■ ■ Erreur fréquente

Si ce fichier contient 'banaction = ufw-multiport', fail2ban refuse de démarrer (action inexistante). Remplacer par le contenu minimal ci-dessus.

5.4 Démarrer et tester

```
bash
sudo systemctl restart fail2ban
sudo fail2ban-client status sshd

# Test : 3 tentatives échouées depuis une autre machine
ssh -p 222 user@192.168.1.3 # entrer 3 mauvais mots de passe
# L'IP doit apparaître dans : Banned IP list
```

CHAPITRE 06

Service systemd d'audit

Créer un service systemd qui exécute en arrière-plan un script Python d'audit système toutes les 5 minutes. Le service redémarre automatiquement en cas de crash et démarre au boot du serveur.

6.1 Créer le script Python

Emplacement standard pour les scripts système : `/usr/local/bin/`

```
• • •  
sudo nano /usr/local/bin/audit.py
```

bash

python

```
#!/usr/bin/env python3
import subprocess, datetime, time, os

LOG = "/var/log/audit.log"

def log(msg):
    ts = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG, "a") as f:
        f.write(f"[{ts}] {msg}\n")

def get_cpu():
    r = subprocess.run(["top", "-bn1"], capture_output=True, text=True)
    for line in r.stdout.split("\n"):
        if "Cpu(s)" in line:
            return line.strip()
    return "N/A"

def get_ram():
    r = subprocess.run(["free", "-h"], capture_output=True, text=True)
    lines = r.stdout.strip().split("\n")
    return lines[1] if len(lines) > 1 else "N/A"

def get_disk():
    r = subprocess.run(["df", "-h", "/"], capture_output=True, text=True)
    lines = r.stdout.strip().split("\n")
    return lines[1] if len(lines) > 1 else "N/A"

def get_ssh():
    r = subprocess.run(["who"], capture_output=True, text=True)
    sessions = [l for l in r.stdout.strip().split("\n") if l]
    return f"{len(sessions)} session(s) active(s)"

while True:
    log("=== Audit système ===")
    log(f"CPU : {get_cpu()}")
    log(f"RAM : {get_ram()}")
    log(f"DISK : {get_disk()}")
    log(f"SSH : {get_ssh()}")
    time.sleep(300) # 5 minutes
```

bash

```
# Rendre le script exécutable
sudo chmod +x /usr/local/bin/audit.py
```

6.2 Créer le fichier service

bash

```
sudo nano /etc/systemd/system/log-python.service
```

```

ini
[Unit]
Description=Script d'audit système toutes les 5 minutes
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/bin/python3 /usr/local/bin/audit.py
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

```

■ Structure d'un service

Le fichier `.service` contient 3 sections : `[Unit]` pour la description et les dépendances, `[Service]` pour comment lancer le programme, `[Install]` pour le comportement au boot. L'extension `.service` est obligatoire — sans elle, `systemd` ne reconnaît pas le fichier.

6.3 Activer et démarrer

```

bash
# Recharger systemd pour qu'il détecte le nouveau fichier
sudo systemctl daemon-reload

# Activer au boot ET démarrer immédiatement
sudo systemctl enable --now log-python

# Vérifier l'état
sudo systemctl status log-python

# Vérifier que le log est généré
sudo tail -f /var/log/audit.log

```

Résultat attendu dans `/var/log/audit.log` :

```

bash
[2026-06-20 14:16:16] === Audit système ===
[2026-06-20 14:16:16] CPU   : %Cpu(s):  0.0 us,  3.3 sy,  0.0 ni, 96.7 id,  0.0 wa
[2026-06-20 14:16:16] RAM   : Mem:    3.8Gi  255Mi  2.8Gi  1.0Mi  836Mi  3.3Gi
[2026-06-20 14:16:16] DISK  : /dev/sda2      7.8G  3.4G  4.1G  46% /
[2026-06-20 14:16:16] SSH   : 2 session(s) active(s)

```

CHAPITRE 07

Rotation des logs (Logrotate)

Sans rotation, `/var/log/audit.log` grossit indéfiniment et finit par saturer le disque. Logrotate compresse et archive automatiquement les anciens logs.

7.1 Créer la configuration

```
bash
sudo nano /etc/logrotate.d/audit
```

```
ini
/var/log/audit.log {
    weekly
    rotate 4
    compress
    missingok
    notifempty
}
```

7.2 Comprendre les options

- › **weekly** — rotation hebdomadaire
- › **rotate 4** — garde 4 versions archivées (4 semaines)
- › **compress** — compresse les anciens logs en .gz
- › **missingok** — ne génère pas d'erreur si le fichier n'existe pas
- › **notifempty** — ne fait pas la rotation si le fichier est vide

7.3 Tester la configuration

```
bash
# Tester sans appliquer (mode debug)
sudo logrotate -d /etc/logrotate.d/audit

# Forcer la rotation pour tester
sudo logrotate -f /etc/logrotate.d/audit

# Vérifier les fichiers archivés
ls -la /var/log/audit.log*
```

CHAPITRE 08

Validation finale

Checklist finale à valider pour considérer le Lab 1 comme terminé :

✓	Réseau	IP statique configurée, ping internet et DNS OK
✓	Users	Utilisateur sudo + utilisateur deployer sans shell
✓	SSH	Port 222 actif, auth clé uniquement, root login désactivé
✓	UFW	Firewall actif, port 22 bloqué, ports 222/80/443 ouverts
✓	Fail2Ban	Service actif avec banaction=ufw, test de ban validé
✓	Systemd	Service log-python actif au boot, /var/log/audit.log alimenté
✓	Logrotate	Configuration créée et testée sans erreur

■ Prochaine étape

Lab 1 terminé. Ce serveur est maintenant prêt à accueillir des services en production (web, API, base de données). Prochaine étape : automatiser cette configuration via Ansible pour pouvoir provisionner 10 serveurs identiques en une seule commande.